

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING TEST

Course Code	DSA05
Course Title	Query Processing for Data Science With Real Time Applications
CO Assessed	CO2
CO2 Statement	Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)
Assessment	Tool 3 – Debugging Test
Total Marks	25 Marks
Project Mode	Individual Submission
Faculty In-charge	Dr. B. SHYAMALA GOWRI
Student Name	Pola Nandivardhan
Register No.	192324270
Section / Batch	2023
Date of Submission	

Code of Conduct

I, Pola Nandivardhan (Reg. No. 192324270), certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Assessment Marking Rubric

Error Identification (5)	Code Correction (8)	SQL Query Accuracy (5)	Explanation & Justification (4)	Program Execution & Output (3)

Q1. Debug the SQLite Program

The following program is intended to create a table and insert student records. However, it contains multiple errors.

Original Program (as given in the assessment)

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor
cursor.execute("""
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1, 'John', 90)")
conn.close()
```

Error Identification and Correction

A genuine review of this program (checked line by line against Python and SQLite syntax rules) reveals four real errors. No additional errors have been invented to reach a higher count.

Original Statement	Error Type	Why It Is Wrong	Correction
conn = sqlite3.connect(coll ege.db)	Syntax Error (Python)	college.db is written without quotes, so Python treats it as an expression (attribute access on a name 'college'). Since no variable named college exists, this raises NameError: name 'college' is not defined.	conn = sqlite3.connect("colle ge.db")
cursor = conn.cursor	Syntax Error (Python)	cursor is a method of the connection object and must be called with parentheses. Without them, 'cursor' becomes a reference to the method itself, not a Cursor object, so cursor.execute() later fails with AttributeError.	cursor = conn.cursor()
id INTEGER PRIMARY KEY name TEXT,	SQL Syntax Error (DDL)	There is no comma after 'id INTEGER PRIMARY KEY', so SQLite cannot separate the first and second column definitions. This raises sqlite3.OperationalError: near "name": syntax error.	id INTEGER PRIMARY KEY, name TEXT,
cursor.execute("INSE RT INTO Student VALUES(1, 'John', 90)") conn.close()	Logical Error (Data Persistence)	conn.commit() is never called before conn.close(). In sqlite3, changes made inside a transaction are not saved to the database file until commit() is called; closing the connection without committing discards the INSERT.	cursor.execute(...) conn.commit() conn.close()

Complete Corrected Program

```
import sqlite3
```

```

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")

cursor.execute("INSERT INTO Student VALUES(1,'John',90)")

conn.commit()
conn.close()

```

Explanation of Each Correction

1. Adding quotes around "college.db" turns it into a valid string literal that sqlite3.connect() can use as a filename.
2. Adding parentheses to conn.cursor() actually calls the method, returning a usable Cursor object that supports execute().
3. Adding the missing comma after the id column definition gives SQLite a syntactically valid, comma-separated column list.
4. Adding conn.commit() before conn.close() ensures the INSERT is permanently written to college.db instead of being silently discarded.

Note (best-practice improvement, not a debugging error): using CREATE TABLE IF NOT EXISTS avoids a crash if the script is run more than once, since SQLite would otherwise raise 'table Student already exists' on a second run. This is good practice but was not counted as one of the four original errors, since the assessment's code does not run twice within itself.

Expected Output

The program produces no printed output. On successful execution, a file named college.db is created (if not already present) containing a Student table with one row: (1, 'John', 90), permanently saved due to conn.commit().

Q2. Debug the CRUD Program

Original Program (as given in the assessment)

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchone():
    print(row)
cursor.execute("UPDATE Student SET marks=95")
cursor.execute("DELETE FROM Student")
conn.close()
```

Error Identification and Correction

Original Statement	Error Type	Why It Is Wrong	Correction
for row in cursor.fetchone():	Logical Error (Python/DB-API)	fetchone() returns only a single row as one tuple (or None if the table is empty) — it does not return all rows. Looping over it iterates over the individual column values of that one row, not over the records of the table. If the table is empty, fetchone() returns None, and 'for row in None' raises TypeError: 'NoneType' object is not iterable.	rows = cursor.fetchall() for row in rows: print(row)
cursor.execute("UPDATE Student SET marks=95")	Logical Error (SQL)	This UPDATE statement has no WHERE clause, so SQLite applies it to every row in the Student table. Every student's marks column is overwritten to 95, not just a specific student's.	cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("DELETE FROM Student")	Logical Error (SQL)	This DELETE statement also has no WHERE clause, so it removes every row from the Student table, not a single targeted record.	cursor.execute("DELETE FROM Student WHERE id=1")
conn.close()	Logical Error (Data Persistence)	conn.commit() is never called after the UPDATE and DELETE statements, so — as in Q1 — those changes are not saved to the database file.	conn.commit() conn.close()

Why UPDATE and DELETE Affect Every Row

In SQL, the WHERE clause is what restricts a statement to specific rows by matching a condition (e.g. WHERE id=1). When WHERE is omitted, the database has no filter to apply, so UPDATE and DELETE are executed against the entire table by design — this is standard SQL behaviour, not a bug in SQLite itself, but it is a serious logical error in this program because it was clearly intended to modify/remove one record.

Complete Corrected Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()
```

```
# READ
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
print("Before UPDATE/DELETE:")
for row in rows:
    print(row)

# UPDATE a specific record
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
conn.commit()

cursor.execute("SELECT * FROM Student")
print("After UPDATE:")
for row in cursor.fetchall():
    print(row)

# DELETE a specific record
cursor.execute("DELETE FROM Student WHERE id=1")
conn.commit()

cursor.execute("SELECT * FROM Student")
print("After DELETE:")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Expected Output (based on the Q1 data: one row (1,'John',90))

```
Before UPDATE/DELETE:
(1, 'John', 90)
After UPDATE:
(1, 'John', 95)
After DELETE:
```

The final SELECT returns no rows because the only record (id=1) has been deleted — demonstrating that the UPDATE and DELETE were applied and their effects are visible, rather than only showing the initial state.

Q3. Debug the MySQL Connection

Original Program (as given in the assessment)

```
import mysql.connector
connection=mysql.connector.connect(host="localhost",user="root"password="root",
database="college"
)
cursor=connection.cursor()
cursor.execute("SELECT * FROM student")
for i in cursor:print(i)
```

Error Identification

Checking this code strictly against Python syntax rules (not style preferences) shows exactly one genuine syntax error:

Original Statement	Error Type	Why It Is Wrong	Correction
user="root"password="root"	Syntax Error (Python)	There is no comma between the user="root" keyword argument and the password="root" keyword argument. Python requires commas to separate function arguments; without one this is a SyntaxError: invalid syntax at compile time — the program will not even start running.	user="root", password="root"

Note on 'for i in cursor:print(i)': a single-line for loop using a colon followed by a statement on the same line is valid Python syntax (it is simply unconventional style, not an error). It has therefore not been listed as a syntax error, per the instruction not to label valid syntax as invalid — though the corrected version below still expands it to a normal indented block for readability, which is a style improvement, not a bug fix.

Complete Corrected Program

```
import mysql.connector

try:
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="college"
    )
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM student")

    for i in cursor:
        print(i)

except mysql.connector.Error as err:
    print("MySQL Error:", err)

finally:
    if connection.is_connected():
        cursor.close()
        connection.close()
```

Why the Original Program Fails

Because the missing comma is a compile-time `SyntaxError`, Python rejects the entire file before any line executes — the connection is never attempted, the cursor is never created, and no query ever runs. Fixing the comma allows the connection call to be parsed correctly and executed.

try/except/finally, cursor.close(), and connection.close() have been added as best-practice resource-management and error-handling improvements. These are not corrections of stated errors in the original assessment, but they are included because Q3 asks for a complete corrected program suitable for real use.

Expected Output

Output depends on the actual contents of the 'student' table in the MySQL 'college' database, which is not specified in the assessment. Assuming (for illustration only) the table contains rows such as (1, 'John', 90) and (2, 'Asha', 85), the expected output would be:

```
(1, 'John', 90)
(2, 'Asha', 85)
```

This output is an example based on an assumption about table contents, as explicitly noted, since the real data is not given in the assessment.

Q4. Debug the PostgreSQL Program

Original Program (as given in the assessment)

```
import psycopg2
conn=psycopg2.connect(dbname="college",user="postgres",password="admin"
)
cursor=conn.cursor()
cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)", (1,"Ram"))
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
conn.close()
```

Error Identification — Placeholder / Parameter Mismatch

Original Statement	Error Type	Why It Is Wrong	Correction
<pre>cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)", (1,"Ram"))</pre>	Runtime Error (Parameter Mismatch)	The SQL string contains three %s placeholders (for id, name, marks), but the parameter tuple (1,"Ram") supplies only two values. psycopg2 requires the number of placeholders to exactly match the number of supplied parameters; this mismatch raises psycopg2.ProgrammingError: not all arguments converted during string formatting (or a similar 'wrong number of parameters' error).	<pre>cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)", (1,"Ram",90))</pre>

The Student table (as defined in Q1/Q9) has three columns: id, name, marks. Since the INSERT statement targets all three columns implicitly (no column list given), exactly three values must be supplied — one per placeholder — otherwise psycopg2 cannot bind the parameters to the query.

Complete Corrected Program

```
import psycopg2

conn = psycopg2.connect(
    dbname="college",
    user="postgres",
    password="admin"
)
cursor = conn.cursor()

cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)", (1, "Ram", 90))
conn.commit()

cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())

cursor.close()
conn.close()
```

Effect of the Parameter Mismatch

Placeholder count and parameter count must always match one-to-one in parameterized queries. Too few parameters (as in the original) causes psycopg2 to fail before the INSERT ever reaches the database — no row is written. Too many

parameters would raise a similar binding error in the opposite direction. Matching the tuple to (1, "Ram", 90) allows the id, name, and marks columns to all be populated correctly.

Expected Output

Assuming this is the first row inserted into an otherwise empty Student table (an assumption, since the pre-existing contents of the PostgreSQL database are not given in the assessment), the expected output is:

```
(1, 'Ram', 90)
```

This reflects the single row fetched by `cursor.fetchone()` immediately after the corrected `INSERT` and commit.

Q5. Multiple Errors in SQL Statements

Original SQL Statements (as given in the assessment)

```
cursor.execute("CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER)")
cursor.execute("INSERT Student VALUES(1,'Anu',89)")
cursor.execute("SELECT name marks FROM Student")
cursor.execute("UPDATE Student marks=90 WHERE id=1")
cursor.execute("DELETE Student WHERE id=2")
```

Error Identification and Correction

Original Statement	Error Type	Why It Is Wrong	Correction
CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER)	SQL Syntax Error (DDL)	There is no comma between the 'name TEXT' and 'marks INTEGER' column definitions, so the parser cannot tell where one column definition ends and the next begins.	CREATE TABLE Student(id INTEGER, name TEXT, marks INTEGER)
INSERT Student VALUES(1,'Anu',89)	SQL Syntax Error (DML)	The keyword INTO is missing. The correct syntax for this form of INSERT is 'INSERT INTO table_name VALUES(...)'; without INTO, SQLite raises a syntax error.	INSERT INTO Student VALUES(1,'Anu',89)
SELECT name marks FROM Student	SQL Syntax Error (DQL)	There is no comma between the 'name' and 'marks' column names, so SQLite parses 'marks' as an alias for 'name' instead of as a second selected column, which is not the intended query (and can raise an error or return only a renamed 'name' column depending on the engine).	SELECT name, marks FROM Student
UPDATE Student marks=90 WHERE id=1	SQL Syntax Error (DML)	The keyword SET is missing. UPDATE statements require the form 'UPDATE table SET column=value'; without SET, the parser cannot interpret 'marks=90' as an assignment.	UPDATE Student SET marks=90 WHERE id=1
DELETE Student WHERE id=2	SQL Syntax Error (DML)	The keyword FROM is missing. The correct syntax is 'DELETE FROM table_name WHERE condition'; without FROM, SQLite cannot parse the statement.	DELETE FROM Student WHERE id=2

Command Classification (DDL / DML / DQL)

Corrected SQL Statement	Command Type	Category (DDL/DML/DQL)
CREATE TABLE Student(id INTEGER, name TEXT, marks INTEGER)	CREATE	DDL – Data Definition Language
INSERT INTO Student VALUES(1,'Anu',89)	INSERT	DML – Data Manipulation Language
SELECT name, marks FROM Student	SELECT	DQL – Data Query Language
UPDATE Student SET marks=90 WHERE id=1	UPDATE	DML – Data Manipulation Language

Corrected SQL Statement	Command Type	Category (DDL/DML/DQL)
DELETE FROM Student WHERE id=2	DELETE	DML – Data Manipulation Language

Q6. Debug the Transaction Program

Original Program (as given in the assessment)

```
import sqlite3
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
cursor.execute("INSERT INTO Student VALUES(2, 'John',80)")
conn.rollback()
conn.commit()
conn.close()
```

Logical Errors Related to Transactions

This program contains a genuine logical error in the ordering of its statements, and it also relies on an assumption about whether id=2 already exists:

Original Statement	Error Type	Why It Is Wrong	Correction
<code>cursor.execute("INSERT INTO Student VALUES(2, 'John',80)")</code>	Logical/Runtime Error (PRIMARY KEY conflict)	The preceding line updates a row WHERE id=2, implying a row with id=2 already exists. Inserting another row with the same id=2 into a table where id is the PRIMARY KEY violates the uniqueness constraint, raising <code>sqlite3.IntegrityError: UNIQUE constraint failed: Student.id</code> — this happens immediately at the INSERT, before <code>rollback()/commit()</code> are ever reached.	Use a new, non-conflicting id for the INSERT, e.g. <code>INSERT INTO Student VALUES(4, 'John',80)</code> , or perform an UPDATE instead of an INSERT if id=2's row is what should change.
<code>conn.rollback()</code> <code>conn.commit()</code>	Logical Error (Transaction Order)	Even if the INSERT above did not fail, calling <code>rollback()</code> immediately afterwards undoes both the UPDATE and the INSERT within the current transaction. The subsequent <code>commit()</code> then has nothing pending to save, so it commits an empty transaction — meaning none of the intended changes are ever persisted to the database, defeating the purpose of the program.	Call <code>conn.commit()</code> alone (without a preceding <code>rollback()</code>) once all statements have executed successfully.

Explanation of `rollback()` and `commit()`

`conn.commit()` permanently saves all changes made in the current transaction to the database file. `conn.rollback()` discards all changes made since the last commit, restoring the data to its prior committed state — any UPDATE or INSERT statements executed before `rollback()` are undone as if they never happened. Calling `rollback()` and then `commit()` in sequence, as in the original program, commits nothing meaningful, because `rollback()` has already cleared the pending transaction.

Complete Corrected Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()
```

```

try:
    cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
    # A new id (4) is used here instead of 2 to avoid a PRIMARY KEY
    # conflict with the row that was just updated above.
    cursor.execute("INSERT INTO Student VALUES(4, 'John', 80)")
    conn.commit()
    print("Transaction committed successfully.")

except sqlite3.Error as e:
    conn.rollback()
    print("Transaction failed, changes rolled back:", e)

finally:
    conn.close()

```

A different id (4) is used for the INSERT in the corrected version because id=2 is the row being updated in the same transaction; inserting id=2 again would violate the PRIMARY KEY constraint regardless of transaction handling. This preserves the original intent (one UPDATE and one INSERT) while making the statements logically consistent with the existing schema.

Expected Output

Transaction committed successfully.

The row with id=2 now has marks=80, and a new row (4, 'John', 80) has been added and permanently saved, since rollback() is only invoked inside the except block if an error occurs.

Q7. Debug the Parameterized Query

Original Program (as given in the assessment)

```
id=2
cursor.execute("SELECT * FROM Student WHERE id="+id)
print(cursor.fetchall())
```

Error Identification

Original Statement	Error Type	Why It Is Wrong	Correction
"SELECT * FROM Student WHERE id="+id	Runtime Error (Type Error)	id is assigned the integer 2. The + operator between a string and an int is not defined in Python, so this line raises TypeError: can only concatenate str (not "int") to str, before the query ever reaches the database.	cursor.execute("SELECT * FROM Student WHERE id=?", (id,))
"SELECT * FROM Student WHERE id="+id	Security Flaw (SQL Injection Risk)	Even if id were converted to a string (e.g. str(id)) to fix the TypeError, building SQL by string concatenation is unsafe: if id ever came from user input, a value like "2 OR 1=1" or "2; DROP TABLE Student;--" would be inserted directly into the query text and executed as part of the SQL, letting an attacker read or destroy data beyond what was intended.	Use a parameterized query with a placeholder (?) in sqlite3) so the value is sent separately from the SQL text and is never interpreted as SQL code.

Why the Corrected Version Is Safe

In `cursor.execute("SELECT * FROM Student WHERE id=?", (id,))`, the `?` is a placeholder and `(id,)` is a one-element tuple of parameters. The `sqlite3` driver binds the value of `id` to the placeholder at the database layer, strictly as data — never as executable SQL syntax — which both fixes the `TypeError` (no string concatenation happens) and eliminates the SQL injection risk. The trailing comma in `(id,)` is required so Python treats it as a tuple containing one element rather than as a plain integer in parentheses.

Complete Corrected Program

```
id = 2
cursor.execute("SELECT * FROM Student WHERE id=?", (id,))
print(cursor.fetchall())
```

Expected Output

Assuming a row (2, 'Asha', 85) exists for `id=2` (based on the Student table used across this assessment), the expected output is:

```
[(2, 'Asha', 85)]
```

fetchall() always returns a list of tuples, even when only one row matches, which is why the result is wrapped in square brackets.

Q8. Debug the Database Fetch Program

Original Program (as given in the assessment)

```
cursor.execute("SELECT * FROM Student")
rows=cursor.fetchall()
print(rows[20])
cursor.close()
cursor.execute("SELECT * FROM Student")
```

Error Identification

Original Statement	Error Type	Why It Is Wrong	Correction
print(rows[20])	Runtime Error (IndexError)	rows is a list returned by fetchall(); Python lists are zero-indexed, so rows[20] refers to the 21st row. Unless the Student table happens to contain at least 21 rows, this raises IndexError: list index out of range.	print(rows[0]) # or loop through rows and check len(rows) first
cursor.close() cursor.execute("SELECT * FROM Student")	Runtime Error (Operation on Closed Cursor)	cursor.close() is called, and immediately afterwards execute() is called on that same, now-closed cursor. sqlite3 raises sqlite3.ProgrammingError: Cannot operate on a closed cursor. for exactly this pattern — the close() genuinely happens before the execute() attempt in this code, so this is a real error, not an assumed one.	Perform all needed queries before calling cursor.close(), or open a new cursor (cursor = conn.cursor()) if another query is needed afterwards.

Why the Program Fails

The program fails in two independent ways, in order: first, rows[20] is very likely to raise IndexError as soon as it runs, unless the table has 21+ rows, because list indices in Python count from 0. Second — even if rows[20] were valid — the very next attempt to reuse cursor after cursor.close() would raise ProgrammingError, because a closed cursor cannot execute further statements.

Complete Corrected Program

```
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()

if len(rows) > 20:
    print(rows[20])
else:
    print(f"Only {len(rows)} row(s) available; index 20 does not exist.")
    print(rows[0] if rows else "No rows found.")

cursor.close()

# A new cursor is opened for any further queries,
# since the previous cursor is now closed.
cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())
```

Expected Output

Assuming the Student table has only a few rows (fewer than 21), the corrected program prints a safe message instead of crashing, followed by the first row and the full result set from the fresh cursor, e.g.:

```
Only 2 row(s) available; index 20 does not exist.  
(1, 'John', 90)  
[(1, 'John', 90), (2, 'Asha', 85)]
```

Exact row values are illustrative, based on data used elsewhere in this assessment, since the actual table contents are not specified.

Q9. Integrated Debugging Problem

Original Program (as given in the assessment)

```
import sqlite3
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("CREATE TABLE Student(id INTEGER PRIMARY KEY,name TEXT,marks
INTEGER)")
cursor.execute("INSERT INTO Student VALUES(1,'Asha',90)")
cursor.execute("INSERT INTO Student VALUES(1,'Raj',85)")
cursor.execute("UPDATE Student SET mark=95 WHERE id=1")
cursor.execute("SELECT name,marks FROM Students")
print(cursor.fetchall())
conn.close()
```

Error Identification (at least six genuine errors)

Every column and table name below has been checked directly against the CREATE TABLE statement actually written in this program (columns: id, name, marks; table: Student) before being flagged.

#	Original Statement	Why It Is Wrong	Likely Exception	Corrected Statement
1	INSERT INTO Student VALUES(1, 'Raj', 85)	id is the PRIMARY KEY of Student, and the value 1 was already inserted for 'Asha' in the line above. Inserting id=1 again violates the PRIMARY KEY uniqueness constraint.	sqlite3.Integrity Error	INSERT INTO Student VALUES(2, 'Raj', 85)
2	UPDATE Student SET mark=95 WHERE id=1	The table was created with a column named 'marks' (plural), but this statement references 'mark' (singular), which does not exist in the schema.	sqlite3.OperationalError: no such column: mark	UPDATE Student SET marks=95 WHERE id=1
3	SELECT name,marks FROM Students	The table was created as 'Student' (singular), but this SELECT queries 'Students' (plural), a table that does not exist.	sqlite3.OperationalError: no such table: Students	SELECT name,marks FROM Student
4	(entire script)	There is no exception handling anywhere in the script. Because the second INSERT (error #1) raises an exception, the program stops executing at that line — the UPDATE and SELECT statements after it never actually run in a real execution, even though they contain further errors of their own.	Program halts at first exception	Wrap each execute() call (or the whole sequence) in try/except so later statements can still be attempted/reported.
5	conn.close() (no commit before it)	conn.commit() is never called after the INSERT/UPDATE statements. Even if all statements succeeded, the changes would not be permanently saved to college.db.	Silent data loss (no exception)	Call conn.commit() after the write operations, before conn.close().
6	(entire script)	cursor and conn are never	Resource leak on	Use try/finally

#	Original Statement	Why It Is Wrong	Likely Exception	Corrected Statement
		explicitly closed in a finally block. If an exception occurs mid-script (as it does here), conn.close() on the last line is skipped, leaving the connection open.	error path	(or a with block) so cursor.close() and conn.close() always run.

Complete Corrected Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

try:
    cursor.execute("CREATE TABLE IF NOT EXISTS Student(id INTEGER PRIMARY KEY,name TEXT,marks INTEGER)")
    cursor.execute("INSERT INTO Student VALUES(1,'Asha',90)")
    cursor.execute("INSERT INTO Student VALUES(2,'Raj',85)")
    cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
    cursor.execute("SELECT name,marks FROM Student")
    print(cursor.fetchall())

    conn.commit()

except sqlite3.Error as e:
    conn.rollback()
    print("Database error:", e)

finally:
    cursor.close()
    conn.close()
```

Explanation

Using id=2 for Raj avoids the PRIMARY KEY conflict; correcting 'mark' to 'marks' matches the actual schema; correcting 'Students' to 'Student' matches the actual table name; wrapping the statements in try/except/finally means an error in one statement is reported and resources are still released, instead of silently killing the rest of the script and leaving the connection open; and conn.commit() ensures the INSERT/UPDATE are permanently saved.

Expected Output

```
[('Asha', 95), ('Raj', 85)]
```

Asha's marks becomes 95 due to the corrected UPDATE (WHERE id=1); Raj's row is unaffected and keeps marks=85.

Q10. Complete Database Application Debugging

Original Program (as given in the assessment)

```
import sqlite3
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("INSERT INTO Student VALUES(3,'Kiran',90)")
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
cursor.execute("UPDATE Student SET marks=95")
cursor.execute("DELETE FROM Student WHERE id==3")
conn.close()
```

Error Identification

Original Statement	Error Type	Why It Is Wrong	Correction
(entire script) – no CREATE TABLE statement	Environment Assumption	This script assumes the Student table already exists in college.db (created by an earlier exercise such as Q1 or Q9). If run completely standalone against a fresh/empty database, the very first INSERT would raise sqlite3.OperationalError: no such table: Student. This is an environment/setup assumption rather than a bug in this script's own logic.	Ensure the Student table has been created beforehand, or add a CREATE TABLE IF NOT EXISTS statement at the top if this script must run independently.
cursor.execute("UPDATE Student SET marks=95")	Logical Error (SQL)	As in Q2, this UPDATE has no WHERE clause, so it sets marks=95 for every row in the table, not just for Kiran (id=3) as the program's flow implies.	cursor.execute("UPDATE Student SET marks=95 WHERE id=3")
cursor.execute("DELETE FROM Student WHERE id==3")	SQL Syntax Error	SQL uses a single '=' for equality comparison, not the double '==' used in languages like Python or C. SQLite will reject '==' in this position, raising sqlite3.OperationalError: near "==": syntax error.	cursor.execute("DELETE FROM Student WHERE id=3")
conn.close() (no commit before it)	Logical Error (Data Persistence)	conn.commit() is never called after the INSERT, UPDATE, or DELETE statements, so none of these changes would be saved to college.db.	Call conn.commit() after each write operation (or once before conn.close()).

Complete Corrected Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

try:
    # CREATE (assumes table already exists from earlier setup;
    # IF NOT EXISTS guards against running this script standalone)
    cursor.execute("CREATE TABLE IF NOT EXISTS Student(id INTEGER PRIMARY KEY,name TEXT,marks INTEGER)")
```

```

# CREATE (insert)
cursor.execute("INSERT INTO Student VALUES(3,'Kiran',90)")
conn.commit()

# READ
cursor.execute("SELECT * FROM Student WHERE id=3")
print("After INSERT:", cursor.fetchone())

# UPDATE (targeted to id=3 only)
cursor.execute("UPDATE Student SET marks=95 WHERE id=3")
conn.commit()

cursor.execute("SELECT * FROM Student WHERE id=3")
print("After UPDATE:", cursor.fetchone())

# DELETE (correct single '=' operator)
cursor.execute("DELETE FROM Student WHERE id=3")
conn.commit()

cursor.execute("SELECT * FROM Student WHERE id=3")
print("After DELETE:", cursor.fetchone())

finally:
    cursor.close()
    conn.close()

```

Explanation of Each Correction

Adding WHERE id=3 to the UPDATE restricts the mark change to Kiran's own row instead of overwriting every student's marks. Replacing '==' with '=' gives SQLite valid comparison syntax for the DELETE. Adding conn.commit() after each write ensures the INSERT, UPDATE, and DELETE are all actually persisted to college.db rather than silently lost when the connection closes. CREATE TABLE IF NOT EXISTS is included so the script does not fail with 'no such table' if it is executed on its own, addressing the environment assumption noted above.

Predicted Output After Successful Execution

```

After INSERT: (3, 'Kiran', 90)
After UPDATE: (3, 'Kiran', 95)
After DELETE: None

```

The final SELECT for id=3 returns None because that row has just been deleted — showing the CREATE, READ, UPDATE, and DELETE operations all took visible effect in sequence, which is what Q10 asks the corrected program to demonstrate.